



University of
Nottingham

UK | CHINA | MALAYSIA

COMP3055 Machine Learning

Coursework

Yuyang ZHANG

20514470

scyzy26@nottingham.edu.cn

December 23, 2025

School of Computer Science
University of Nottingham Ningbo China

1 Introduction

Image classification is a fundamental problem in computer vision and serves as a benchmark task for evaluating different machine learning models. It has broad applications in areas such as object recognition, autonomous driving, and intelligent surveillance. In this coursework, experiments are conducted on the CIFAR-10 dataset, which consists of 60,000 colour images of size 32×32 evenly across 10 object categories. The dataset presents challenges including low resolution, high variability within each class, and visual overlap between classes (e.g., cats vs. dogs), making CIFAR-10 an ideal testbed for comparing classical machine learning methods. When working with high-dimensional image data, traditional methods like MLPs typically require manual feature engineering or dimensionality reduction (such as PCA), while CNNs can learn hierarchical features directly from raw pixels. Understanding the trade-offs between these two approaches remains an important practical consideration in machine learning.

Using 5000 - 10000 subjects from the dataset, the goal of this coursework is to investigate and compare two machine learning approaches for CIFAR-10 image classification: a Multi-layer Perceptron (MLP) operating on PCA-reduced features, and a Convolutional Neural Network (CNN) operating on raw image data. The comparison evaluates classification performance (accuracy and F1 score) and computational cost (training time), while also analyzing the degree of overfitting by examining the train-test performance gap. Specifically, this work first examines how PCA dimensionality reduction at different variance retention levels (10%, 30%, 50%, 70%, 80%, 90%, 100%) affects MLP classification performance. The MLP is then optimized through hyperparameter tuning (hidden layer size and learning rate). Another method, a three-layer CNN, is designed to directly extract spatial features from the images without dimensionality reduction, with hyperparameter tuning conducted on training sample size, learning rate, and channel width. Finally, the best configurations of both approaches are compared to determine which method offers better performance for this task.

2 Dataset and Processing

The CIFAR-10 dataset is loaded using PyTorch with its standard split of 50,000 training and 10,000 test images. Images are converted to tensors and normalized to $[0, 1]$. To

reduce computational cost, 5,000 images are randomly sampled from the training set using a fixed seed (seed = 42) for reproducibility. This subset forms the baseline for both MLP and CNN experiments. In Task 3, CNN training is extended to 10,000 images to assess performance scaling. All evaluations use the full 10,000-image test set.

3 Task1: PCA Feature Extraction and Analysis

Principal Component Analysis (PCA) is applied to reduce the dimensionality of the flattened 3,072-dimensional image vectors. The implementation uses scikit-learn’s PCA with full SVD solver and a fixed random state (random_state = 42) for reproducibility. To prevent information leakage, PCA is fitted exclusively on the training subset while the learned transformation is then applied to both training and test sets. Seven PCA configurations are created by specifying different variance retention levels: 10%, 30%, 50%, 70%, 80%, 90%, and 100%. For the 100% case, all 3,072 components are retained, effectively serving as a non-PCA baseline. The transformed features for each configuration are saved to disk as NumPy arrays, which are later loaded for MLP training in Task 2.

The coursework description suggests testing at 10%, 30%, 50%, 70%, and 100% variance retention. However, this creates a large dimensionality gap between 70% (14 components) and 100% (3,072 components). To better analyze performance trends across the full dimensionality range, two additional levels at 80% and 90% are included, yielding intermediate dimensionalities of 32 and 95 components, respectively.

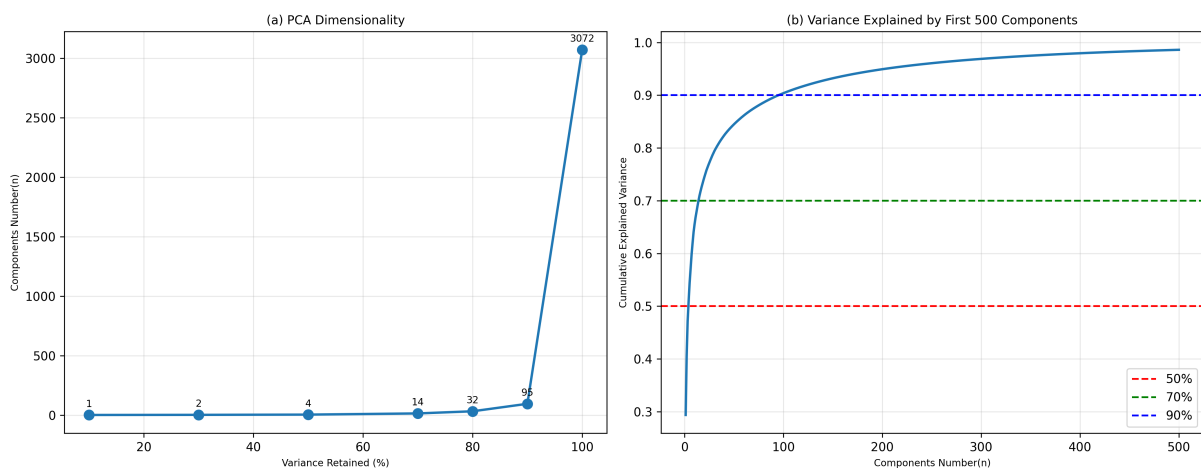


Figure 1: PCA analysis on CIFAR-10 (training subset). (a) Number of principal components required to reach different variance-retention levels. (b) Cumulative explained variance as a function of retained components (first 500), with reference thresholds at 50%, 70%, and 90%.

Figure 1(a) shows the number of components required at each retention level. The relationship is highly non-linear: low retention levels (10%–50%) require very few components (1–4), indicating that dominant variance is concentrated in the leading principal directions. Beyond 70%, dimensionality increases rapidly (32 components for 80%, 95 for 90%, and 3,072 for 100%), showing that additional variance is distributed across many more dimensions with diminishing marginal contribution per component. Figure 1(b) presents the cumulative variance explained by the first 500 components. The curve rises steeply initially, then gradually flattens. Approximately 50% variance is captured within the first few components, 70% requires around 14 components, and 90% needs roughly 95 components. This confirms that most variance concentrates in early components while high retention levels demand substantially more dimensions.

4 Task 2: MLP

4.1 Approach

A Multilayer Perceptron (MLP) is trained using the PCA-reduced features from Task 1. The implementation uses scikit-learn’s MLP Classifier with ReLU activation and the Adam optimizer. Five-fold stratified cross-validation is conducted on the training subset to evaluate performance while preserving class balance across folds. Two sets of experiments are performed:

1. **PCA dimensionality analysis:** The MLP is trained on all seven PCA feature sets (10%, 30%, 50%, 70%, 80%, 90%, 100% variance retention) using a fixed configuration.
2. **Hyperparameter tuning:** Using the original 3,072-dimensional features (100% variance), three configurations are tested by varying hidden layer sizes and learning rates:
 - h256_lr3e-3: hidden size 256, learning rate 0.003
 - h512_lr3e-3: hidden size 512, learning rate 0.003
 - h512_lr1e-3: hidden size 512, learning rate 0.001

All models are trained for 30 epochs. Performance is evaluated using overall and per-class accuracy and F1 score.

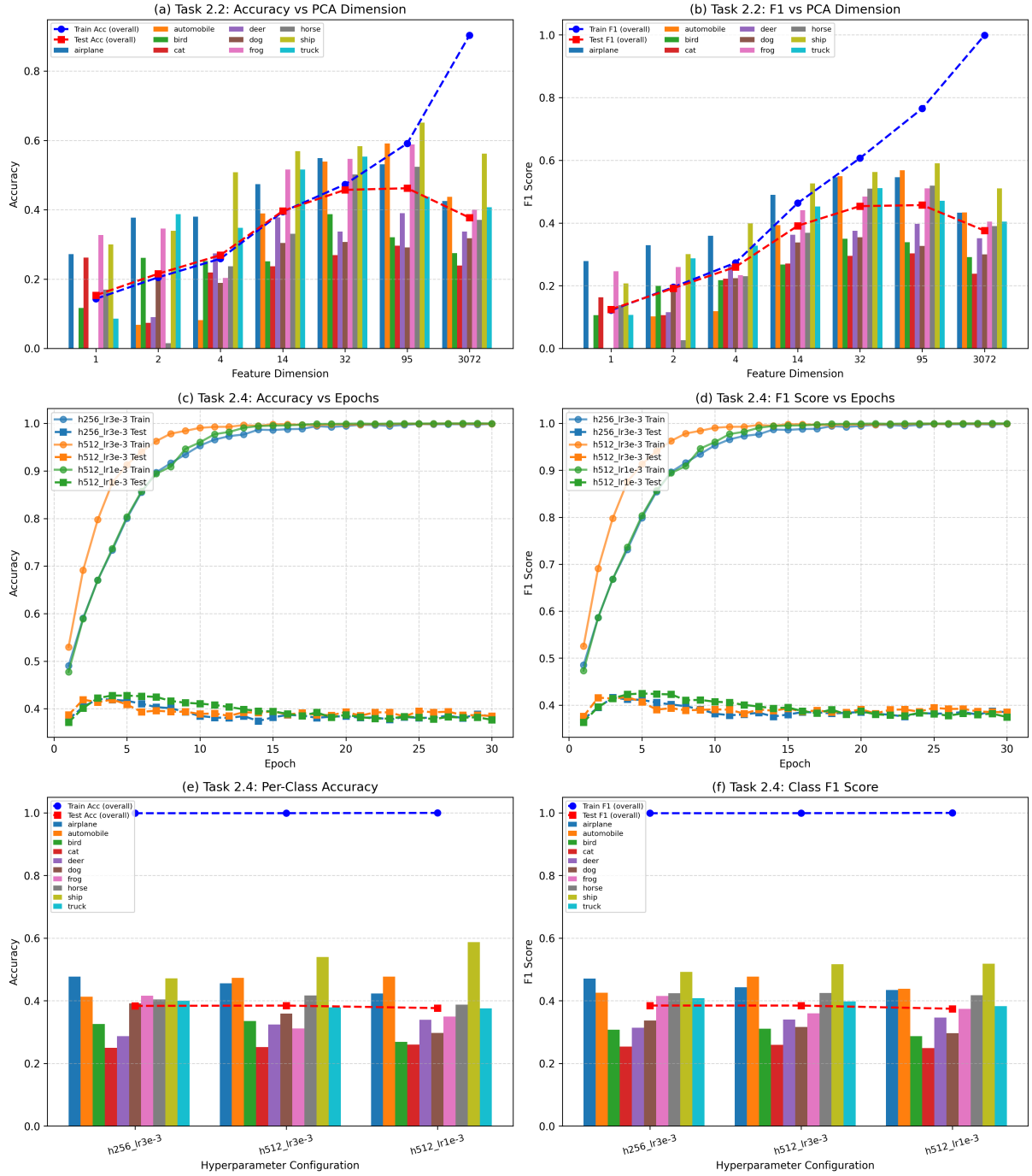


Figure 2: Task 2 results using MLP. (a) Overall and per-class accuracy vs PCA dimensionality. (b) Overall and per-class F1 score vs PCA dimensionality. (c) Training and test accuracy vs epochs for different hyper-parameter settings. (d) Training and test F1 score vs epochs. (e) Per-class accuracy under different hyper-parameters. (f) Per-class F1 score under different hyper-parameters.

4.2 Effect of PCA Dimensionality

Figures 2(a) and (b) show how PCA feature dimensionality affects classification performance. Both training and test accuracy/F1 increase steadily as dimensionality grows from 1 to 95 components, indicating that higher variance retention preserves more discriminative information. At dimensionality $n = 32$, a gap between training and test performance begins to appear and widens at higher dimensionalities. Most severely, at full dimensionality (3,072 components), training performance approaches 100% while test performance drops to approximately 40%, significantly lower than the about 50% achieved at 32 components. This large train-test gap indicates severe overfitting when the MLP operates on the complete high-dimensional input. The pattern is consistent between accuracy and F1 metrics. Moreover, per-class results reveal substantial performance differences across object categories. At very low dimensionality (1-4 components), automobile, deer, and dog show near-zero accuracy and F1, while others maintain moderate performance. As dimensionality increases from low to moderate levels (up to 95 components), all classes show an overall growth trend but at different rates, with ship and frog consistently outperforming others. This suggests that certain object categories are inherently easier to classify using flattened pixel features, while others require more complex representations that the MLP struggles to learn.

4.3 Hyperparameter Tuning

To identify a better MLP configuration, additional experiments are conducted with different hyperparameters. Results are visualized in Figures 2(c) and (d), which show training dynamics across 30 epochs for three configurations using the full 3,072-dimensional features. All configurations show similar behavior: training accuracy and F1 increase rapidly within the first 5 epochs, reaching near-perfect values (about 0.95-1.0) by epoch 10 and remaining stable through epoch 30. In stark contrast, test performance starts low (approximately 0.4), peaks around epochs 3-4, then begins to decrease before plateauing around 0.38-0.40. Similar to the patterns observed in Figures 2(a) and (b), a large train-test gap emerges early in training and continues to widen, indicating severe overfitting regardless of hyperparameter choice. Interestingly, all three configurations converge to nearly identical test performance despite differences in hidden layer size (256 vs 512) and learning rate (0.001 vs 0.003). The main difference between configurations is the convergence speed during the first 10 epochs, suggesting that these hyperparameters primarily

affect optimization dynamics rather than final generalization ability. The consistency between accuracy and F1 curves confirms that the overfitting behavior is not driven by any particular metric or class distribution artifact, but reflects an overall limitation of the MLP when operating on high-dimensional flattened image features.

4.4 Per-class Performance across Different Hyper-parameters

Finally, hyperparameter performance is examined at the class level. Figures 2(e) and (f) compare per-class accuracy and F1 scores for the three configurations. Training performance (blue dashed line) remains near-perfect (about 1.0) across all configurations, while test performance (red dashed line) stays around 0.38-0.40, consistent with the training dynamics observed in Figures 2(c) and (d). At the class level, substantial and consistent disparities are evident. Airplane, automobile, and ship achieve the highest test scores (approximately 0.45-0.50), while cat remains the lowest-performing class (around 0.25-0.30). Other classes like deer and dog also show relatively weak performance. Importantly, the relative performance ranking remains largely consistent across all three configurations: classes that perform well under one setting continue to perform well under others, and weak classes remain weak. The per-class bars shift only slightly between configurations, indicating that changing hidden layer size or learning rate does not fundamentally alter which classes are easy or difficult to classify. This suggests that the MLP’s performance is constrained by the inherent separability of different object categories in the flattened feature space, rather than by insufficient model capacity or suboptimal training parameters. The consistency between accuracy and F1 patterns at the class level further confirms that these differences reflect genuine classification difficulty rather than metric-specific artifacts.

4.5 Summary

Task 2 demonstrates two key findings. First, from the PCA experiments (Section 4.2), moderate dimensionality reduction (80-90% variance retention) achieves better test performance than using full features, indicating that dimensionality reduction might help control overfitting(Section 4.1-4.2). Second, from the hyperparameter tuning experiments (Sections 4.3-4.4), varying model capacity and learning rate on the full-dimensional features provides minimal improvement, with all configurations showing severe overfitting and similar final test performance. Moreover, class-wise analysis reveals persistent performance variation across all experiments. These findings suggest severe limitations exist

with MLP.

5 Task 3: CNN

5.1 Approach

A Convolutional Neural Network (CNN) is designed as an alternative to the MLP approach. Unlike the MLP, which operates on flattened features, the CNN processes images in their original $32 \times 32 \times 3$ tensor format to preserve spatial structure. The CNN architecture consists of three convolutional blocks. Each block contains a convolutional layer, batch normalization, ReLU activation, and max pooling for spatial downsampling. Channel width increases progressively across blocks to learn increasingly abstract features. A dropout layer is applied before the final fully connected classification layer. Data augmentation (random horizontal flip and random crop) is used during training to improve robustness. Similarly to MLP, to identify the best configuration, four variants are tested:

- **base:** Baseline with 5,000 training samples
- **more_samples:** Increased to 10,000 training samples
- **low_learning_rate:** Base configuration with reduced learning rate
- **wider_channel:** Base configuration with increased channel width

All models are trained for 30 epochs using cross-entropy loss and the Adam optimizer. Performance is evaluated on the full test set ($n = 10000$) using accuracy and macro-averaged F1 score.

5.2 Comparison of CNN Configurations

Figures 3(a) and (b) compare overall and per-class accuracy and F1 scores across the four CNN configurations. Training performance (blue dashed line) remains consistently high (approximately 0.9) for all configurations, indicating that the CNN has sufficient capacity to fit the training data. Test performance (red dashed line) ranges from 0.75 to 0.84, showing much better generalization. The `more_samples` configuration achieves the highest test accuracy and F1 (about 0.83-0.84), outperforming the base model as well as the variants with adjusted learning rate or channel width. This improvement is consistent across most classes rather than being driven by a few specific categories, suggesting that increased data availability enhances generalization uniformly. In contrast, the `low_learning_rate` and

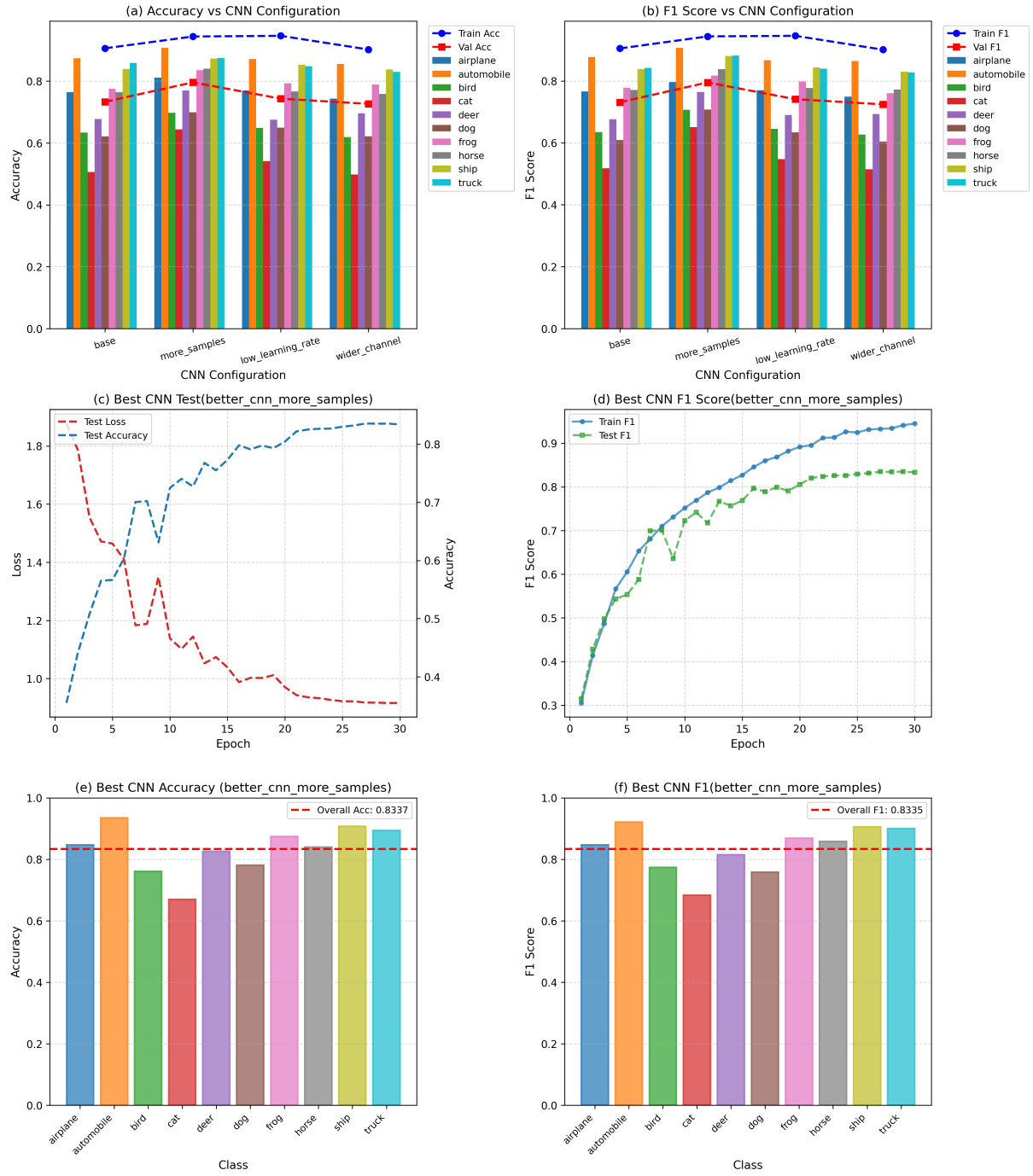


Figure 3: Task 3 CNN results. (a) Overall and per-class accuracy across CNN configurations. (b) Overall and per-class F1 score across CNN configurations. (c) Test loss and test accuracy over epochs for the best CNN configuration. (d) Training and test F1 score over epochs for the best CNN configuration. (e) Per-class accuracy of the best CNN. (f) Per-class F1 score of the best CNN.

wider_channel configurations yield only marginal gains over the baseline, with test performance remaining around 0.75-0.78. This indicates diminishing returns from architectural or optimization adjustments when training data remains limited. Notably, the variance among different classes remains similar across all configurations. Adjusting learning rate or channel width does not fundamentally change the relative performance ranking, which stays consistent. At the class level, performance is more balanced compared to the MLP. While some disparity exists, with classes like airplane and automobile performing slightly better, no classes show the extremely poor performance (about 0.25) observed in the MLP experiments. This suggests that convolutional processing better captures discriminative patterns across all object categories.

5.3 Training Dynamics of the Best CNN Model

To examine the learning behavior of the best configuration (more_samples), Figures 3(c) and (d) show performance evolution across 30 training epochs. Figure 3(c) shows that test loss decreases sharply during the first 10 epochs, then continues to decline more gradually throughout training. Test accuracy correspondingly rises from an initial low value (around 0.3) to a stable high level (about 0.83), showing an inverse relationship with loss that indicates consistent optimization. Although fluctuations in test accuracy appear throughout epochs 8-30, the overall trend remains upward with no signs of instability or performance collapse. Figure 3(d) shows that both training and test F1 scores increase steadily across epochs, with the two curves remaining largely parallel. The gap between training and test F1 stays relatively limited (approximately 0.1) and stabilizes after epoch 20, suggesting that the model continues to improve without developing the severe divergence characteristic of overfitting. This contrasts sharply with the MLP behavior in Task 2, where training metrics quickly approached perfection while test performance stagnated. Both test accuracy and test F1 begin to plateau after approximately 20-25 epochs, with only marginal improvements thereafter. This indicates diminishing returns from continued training and suggests that early stopping around this point could reduce computational cost while preserving nearly identical final performance.

5.4 Per-Class Performance of the Best CNN

Figures 3(e) and (f) show per-class accuracy and F1 scores for the best-performing CNN configuration (more_samples). The model achieves an overall test accuracy of approximately 0.83 and an overall macro-F1 of approximately 0.83 (red dashed line), indicating

strong performance across classes. Classes like airplane, automobile, frog, ship and truck outperform, achieving 0.85-0.9 , while cat remains the weakest class at approximately 0.67-0.7. Other classes like bird and deer also show slightly lower performance compared to the top-performing categories. Despite this variation, the per-class distribution is considerably more concentrated around a high mean compared to the MLP results in Task 2, and no extreme failure cases are observed. The pattern is highly consistent between accuracy and F1: classes with high accuracy also achieve high F1 scores, and lower-performing classes show reduced scores in both metrics, suggesting that the observed differences reflect class-dependent difficulty rather than artifacts of a single evaluation metric.

5.5 Summary

Task 3 demonstrates three key findings. First, from the configuration comparison (Section 5.2), increasing the training set size to 10,000 samples leads to the largest improvement in test performance, indicating that data availability plays a critical role in enhancing CNN generalization. Second, variations in learning rate and channel width provide only marginal gains when training data remains limited, with all configurations showing similar class-wise performance patterns. Moreover, analysis of training dynamics and per-class results (Sections 5.3-5.4) reveals stable optimization behavior with a relatively small train-test gap and consistently strong performance across object categories. These findings suggest that the CNN effectively learns robust representations from image data under appropriate training conditions.

6 Task 4: Comparison and Discussion

6.1 Approach

Based on the best-performing configurations identified in Tasks 2 and 3, the MLP and CNN (with `more_samples` configuration) are compared. The comparison evaluates three aspects:

- Classification performance measured by test accuracy and F1 score for each class
- Training dynamics measured by accuracy and F1 on both training and test sets throughout epochs
- Overfitting behavior and computational cost

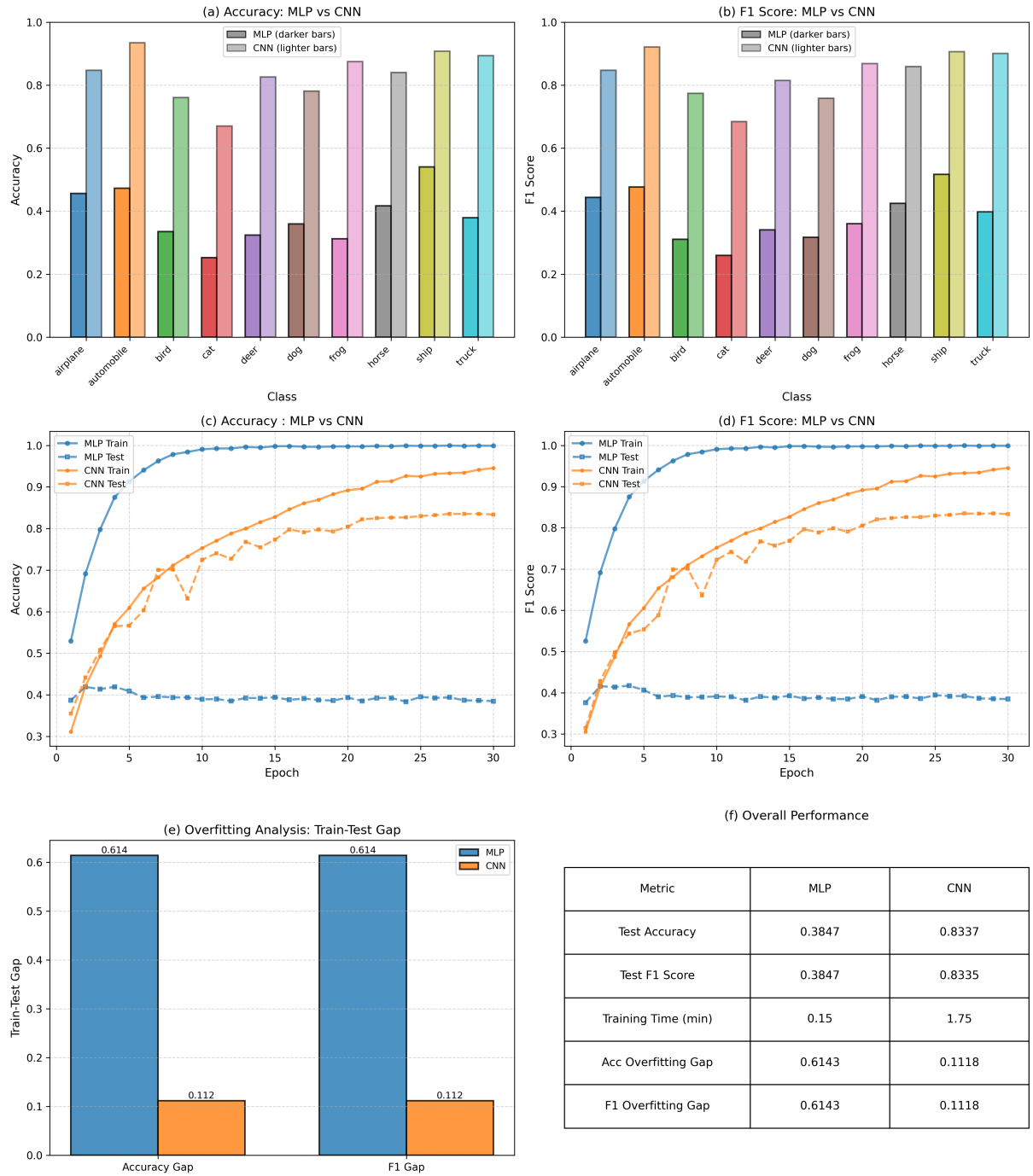


Figure 4: Comparison between MLP (Task 2) and CNN (Task 3). (a) Per-class test accuracy (MLP: darker/solid bars; CNN: lighter bars). (b) Per-class test F1 (MLP: darker/solid bars; CNN: lighter bars). (c) Accuracy vs epoch for train/test sets. (d) F1 vs epoch for train/test sets. (e) Overfitting measured by train-test gap. (f) Overall performance summary including test metrics, training time, and overfitting gaps.

6.2 Per-Class Performance Comparison

Figures 4(a) and 4(b) compare per-class test accuracy and F1 scores between the MLP (darker bars) and CNN (lighter bars). The CNN achieves consistently high scores across all classes, with most in the 0.75-0.90. In contrast, the MLP exhibits much lower performance, with several classes below 0.4. The performance gap varies substantially by class. For classes where the MLP struggled most (such as cat (~ 0.25), dog (~ 0.35), deer (~ 0.32), and bird (~ 0.33), the CNN achieves improvements, reaching 0.67-0.78. Classes where the MLP performed moderately well, such as airplane (~ 0.45), automobile (~ 0.47), and ship (~ 0.53), still see substantial gains with the CNN, rising to 0.85-0.93. Notably, the variance among different classes remains through two different methods. Comparing Figure 4(a) and 4(b), the patterns are highly consistent between accuracy and F1: classes with low MLP accuracy also have low MLP F1, and the CNN improves both metrics together.

6.3 Training Dynamics and Overfitting Analysis

Figures 4(c) and 4(d) compare training and test performance across 30 epochs for both models. The MLP and CNN show different learning behaviors. For the MLP, training accuracy and F1 reaches near-perfect values (about 0.95-1.0) in the first 5 epochs and remains stable throughout training. In contrast, test performance stays flat around 0.38-0.40. This large and persistent train-test gap indicates severe overfitting: the MLP memorizes training data rather than learning generalizable patterns. Meanwhile, the CNN shows different behavior. Both training and test curves increase steadily and remain largely parallel throughout training. Training performance reaches approximately 0.95, while test performance stabilizes around 0.83. Importantly, the train-test gap remains small (around 0.1) and does not widen over time, indicating that the CNN's better performance. To better visualize the difference, Figure 4(e) quantifies the overfitting severity. The MLP shows an accuracy/F1 0.61, while the CNN exhibits much smaller gaps of 0.11, suggesting that the CNN's better performance on preventing overfitting.

6.4 Computational Complexity

Figure 4(f) provides a summary table comparing both methods across test accuracy, test F1, training time per epoch, and train-test overfitting gaps. The table shows that the MLP requires 0.15 minutes on average per epoch while the CNN requires 1.75 minutes

per epoch, roughly 12 times longer, as expected.

6.5 Summary

Task 4 highlights clear differences between the two approaches under different conditions. First, the CNN consistently outperforms the MLP in terms of test accuracy, F1 score, and class-wise stability (Sections 6.2-6.3), indicating that it is a more effective approach when high classification performance and good generalization are required. The CNN maintains a small and stable train-test gap, suggesting strong resistance to overfitting. Second, the MLP demonstrates substantially lower computational cost, with much shorter training time per epoch (Section 6.4), making it a more efficient choice when computational resources are limited or when rapid prototyping is required. However, this efficiency comes at the cost of severe overfitting and poor generalization performance. Overall, for this image classification task on CIFAR-10, the CNN is the better approach due to its superior generalization performance and robustness across object categories.

7 Conclusion

This coursework evaluated MLP and CNN models for image classification on the CIFAR-10 dataset. Results show that while PCA can partially mitigate overfitting in MLPs, their performance remains limited on high-dimensional image data, and gains from hyperparameter tuning are marginal. Overall, CNN-based models demonstrate superior generalization ability and more stable training behavior compared to MLP under the current experimental conditions. With greater computational resources and larger datasets, the CNN's advantage would likely become even more pronounced.